Google Summer of Code 2026

Improving Kubeflow Pipelines UI Support for Pod Lifecycle Failures

Organization: Kubeflow

Project Focus: Kubeflow Pipelines UI

Technology Stack: React • TypeScript • Kubernetes • Argo

Applicant Information

Name: Papana Ratnapradeep Reddy

Email: prpr23091999@gmail.com

GitHub: github.com/RATNAPRADEEP

LinkedIn: linkedin.com/in/ papanaratnapradeepreddy

Time zone: IST (UTC +5:30)

Abstract (Executive summary)

Kubeflow Pipelines enables scalable ML workflows on Kubernetes, but debugging pipeline failures remains complex and time-consuming due to reliance on raw logs and manual inspection. This proposal introduces a frontend error classification panel that converts Kubernetes pod failures into structured, actionable insights directly within the UI. By leveraging multi-signal validation (pod status, exit codes, and log patterns), the system provides accurate error categorization, root cause explanations, and suggested fixes.

A working prototype has already been developed and validated against real failure scenarios such as OOMKilled, CrashLoopBackOff, and ImagePullBackOff, demonstrating feasibility within the existing Kubeflow frontend architecture.

The proposed solution improves observability, reduces debugging effort, and enhances developer productivity while maintaining strict architectural boundaries by avoiding backend or workflow engine modifications.

Proof of Work (Prototype)

-> Developed a working frontend prototype for Kubernetes pod failure classification

- > Supports OOMKilled, CrashLoopBackOff, ImagePullBackOff

- > Provides concise summaries output: Error Type, Root Cause, Suggested Fix

- > Demonstrates feasibility of UI-based error summarization

GitHub: https://github.com/RATNAPRADEEP/Kubeflow-ui-error-analysis

Prototype / Demo

(Refer to GitHub repository above)

Prototype tested with real pipeline failure scenarios. The prototype closely mirrors the proposed final UI integration and validates feasibility within the existing Kubeflow frontend architecture.

Problem Statement

Kubeflow Pipelines enables scalable and reproducible ML workflows, but debugging pipeline failures is currently inefficient and cognitively demanding. When a pipeline step fails, users must:

- Manually navigate through workflow nodes
- Inspect multiple pod logs
- Identify the failing container
- Scan large volumes of unstructured log output
- Infer the root cause without structured guidance

Although Kubeflow exposes raw logs and workflow status, it does not provide clear and structured error summaries, root-cause summarization, or intelligent prioritization of errors. As workflows grow in complexity with parallel components and distributed execution, diagnosing failures becomes increasingly difficult.

This lack of structured error interpretation leads to:

- Increased debugging time
- Reduced developer efficiency
- Steeper onboarding curve for new contributors
- Higher operational friction in ML experimentation

There is a clear need for a system that transforms raw workflow failure signals into concise, categorized, and clear diagnostics directly within the user interface.

This proposal addresses the Kubeflow GSoC project idea “Enhance Support for Pod Lifecycle Failure” by improving the Kubeflow Pipelines UI to surface clear explanations for pod lifecycle failures such as ImagePullBackOff, CrashLoopBackOff, and OOMKilled.

Example Failure Scenario

Consider a pipeline step that fails due to memory exhaustion.

Currently, a user must:

- Navigate to the failed node
- Open the corresponding pod logs
- Scroll through dense log output
- Notice exit code 137 or OOMKilled
- Infer that container memory limits were insufficient

With the proposed system, the UI would instead display:

Error Type: ResourceLimitError

Root Cause: Container terminated due to OOMKilled (exit code 137)

Suggested Fix: Increase memory request/limit in component specification

This reduces multiple manual debugging steps to a single actionable insight

UI Mockup

⚠ Pipeline Step Failed

Error Type: ResourceLimitError

Root Cause: Container terminated (OOMKilled)

Suggested Fix:

Increase memory request/limit in component spec.

[View Full Logs] [Possible Causes ▼]

Related Work

Several tools and mechanisms exist for debugging failures in Kubernetes-based systems. Kubernetes itself exposes failure signals through container exit codes, pod status messages, and events such as OOMKilled or ImagePullBackOff.

Argo Workflows, which Kubeflow Pipelines uses as its execution engine, provides workflow node status and log access for debugging failed steps. However, these mechanisms primarily expose raw infrastructure signals and require manual inspection by users.

Kubeflow Pipelines currently provides access to workflow logs and status information through the RunDetails interface, but it does not provide structured failure interpretation or root-cause summarization.

The proposed work builds upon these existing mechanisms by introducing a deterministic rule-based classification layer that translates Kubernetes and Argo failure signals into concise, actionable debugging insights directly within the Kubeflow Pipelines UI.

Proposed Solution

To reduce debugging friction in Kubeflow Pipelines, this project implements a frontend error classification system integrated at the Kubeflow Pipelines UI layer using existing RunService APIs without modifying backend services or Argo controllers. The design prioritizes incremental PRs to align with Kubeflow's review process and reduce integration risk.

The system introduces a structured error summary layer between Argo workflow execution and the UI.

To ensure reliable and accurate classification, the system uses a multi-signal validation strategy. Instead of relying on a single indicator such as exit codes, it correlates Kubernetes pod status, container termination reasons, and log patterns to identify the most probable root cause. Unlike single-signal approaches that rely only on exit codes or raw logs, this system correlates pod status, termination reasons, and log patterns to reduce false positives. This improves classification precision and prevents misleading summaries in ambiguous cases such as exit code 137.

The classification engine is designed to handle real-world ambiguity effectively. For example, OOMKilled errors may arise from insufficient memory limits, memory leaks, or large batch sizes. The system surfaces these as multiple possible causes, improving debugging guidance.

Additionally, the rule registry will include patterns for runtime dependency failures such as missing libraries, module import errors, and environment misconfigurations. This ensures coverage of both infrastructure-level and application-level failures commonly observed in ML workflows.

The system is intentionally designed to be deterministic, transparent, and extensible, enabling contributors to safely add new rules without modifying core logic.

Core Innovation

- Rule-based, pluggable Error Rule engine.
- Two-tier classification: Kubernetes status mapping + 1MB tail log parsing.
- Confidence scoring to minimize false positives.
- Sidecar-aware filtering (main container logs only).
- Backward-compatible, frontend-driven integration

The implementation will begin with a minimal set of high-confidence rules (e.g., OOMKilled, CrashLoopBackOff, ImagePullBackOff) and incrementally expand to additional failure types.

Technical Design

The implementation is strictly integrated with the Kubeflow Pipelines RunDetails UI component to surface pod lifecycle failure explanations directly within the pipeline execution view.

1. **Understanding of Pipeline Architecture**

Kubeflow Pipelines operates as a multi-layer system consisting of the Python SDK, Go backend (API Server, Driver, Launcher), Argo Workflows execution layer, persistence components, and a React-based frontend.

Argo Workflow

↓

Kubernetes Pod Status + Logs

↓

Signal Extraction Layer

↓

Error Classification Engine

↓

Rule Registry Evaluation

↓

Error Summary Object

↓

Kubeflow UI (ErrorSummaryPanel)

The system integrates strictly at the frontend layer, preserving execution semantics and avoiding changes to Argo controllers, backend services, or persistence logic.

2. Data flow of classification

User opens RunDetails

↓

RunDetails.tsx fetches pod status + logs

↓

LogViewer provides tail log string

↓

classifyError(podStatus, tailLogs)

↓

ErrorRule registry evaluated (priority sorted)

↓

Highest-confidence match selected

↓

ErrorSummaryBanner rendered Classification

Complexity:

O(n) where n = number of rules (bounded, small).

3. Technical integration plan

The Error Classification Engine will integrate at the following concrete frontend locations:

- *frontend/src/pages/RunDetails.tsx* – Inject summary rendering into node failure view.
- *frontend/src/components/LogViewer.tsx* – Use existing log-fetching logic and extract tail logs (≤1MB).
- *frontend/src/components/NodeDetailsPanel.tsx* – Render ErrorSummaryBanner above raw logs.
- *frontend/src/apis/* – Reuse existing RunService and log APIs without backend modification

The classifier will consume:

- Pod phase (FAILED / ERROR)
- Container termination reason
- Exit code
- Tail log string

No changes will be made to:

- *run_server.go*
- Argo workflow controller
- Persistence-agent
- Database schema

This ensures architectural containment and zero execution-layer regression risk.

4. Example rule sample

```
const OOMKilledRule: ErrorRule = {
 id: "OOM_KILLED",
 pattern: /OOMKilled|exit code 137/i,
 category: "Resource",
 severity: "High",
 confidence: 0.95,
 suggestion: "Container exceeded memory limits. Increase memory request/limit in component
 spec."
 };
```

<u>Implementation Details</u>

Classification is performed on the final segment of logs (last 200–500 lines) to efficiently capture terminal failure signals while maintaining performance. The process runs asynchronously in the frontend to avoid blocking UI rendering.

The rule engine follows a deterministic evaluation strategy with priority-based ranking and confidence scoring. In cases of ambiguity, multiple possible causes are presented to the user to improve transparency and avoid misleading conclusions.

1. **Failure signal Extraction:**

When a run enters Failed phase:

Identify terminal failed nodes from workflow.status.nodes.

Map node → Kubernetes pod.

Retrieve:

Container termination reason

Exit code

Last log segment

Kubernetes status message

Focus on high-signal fields:

OOMKilled

ImagePullBackOff

CrashLoopBackOff

ExitCode!= 0

Extracted signal structure:

```
{
 stepName,
 podName,
 exitCode,
 k8sReason,
 logSnippet,
 timestamp
}
```

2. **Rule Registry Extensibility**

Error rules will be stored in a dedicated errorRules.ts module with:

- Clear categorization
- Inline documentation
- Unit tests per rule
- Versioned additions

Future contributors can add rules without modifying classifier logic.

A CONTRIBUTING_ERRORS.md guide will document how to safely extend the registry

Example workflow for adding a new rule:

1. Add a new entry in errorRules.ts with category, severity, and suggestion.
2. Write a unit test in errorRules.test.ts to validate detection.
3. Document the rule in CONTRIBUTING_ERRORS.md with rationale and usage notes.

This ensures consistent rule additions without modifying core classifier logic.

Introduce a deterministic classification layer.

Create structured error categories:

- InfrastructureError
- ImagePullError
- DependencyError
- PermissionError
- ResourceLimitError
- ConfigurationError
- RuntimeCrash

Initial Scope

The initial implementation focuses on common Kubernetes pod lifecycle failures including ImagePullBackOff, CrashLoopBackOff, and OOMKilled.

• OOMKilled – Container terminated due to memory limit exceeded

• ImagePullBackOff – Failure to pull container image from registry

• CrashLoopBackOff – Container repeatedly crashes during runtime

Additional classifiers can be added incrementally through the extensible rule registry after the core framework is established.

Future extensions may include configurable timeout detection for stalled pods and visualization of provisioning-level failures such as Unschedulable.

These failures correspond to common Kubernetes pod lifecycle failures referenced in the Kubeflow GSoC project “Enhance Support for Pod Lifecycle Failure”.

The system will also support configurable timeout detection and UI visualization so that stalled pods or long-running failure states are clearly surfaced in the Kubeflow Pipelines interface.

Each classifier defines:

- Trigger rules (k8s reason, exit code, regex pattern)
- Confidence scoring logic
- Suggested remediation hint
- UI visualization of classified pod lifecycle failures within the Kubeflow Pipelines RunDetails interface.

Handling Ambiguous Exit Codes (Disambiguation Logic)

"While Kubernetes exit codes provide a starting signal, they are often polymorphic. To ensure high precision in the Summary Panel, the Classification Engine will employ a Multi-Signal Verification strategy for ambiguous codes:

- Exit Code 137 (SIGKILL):
    - *Ambiguity:* Can be triggered by OOMKilled, a failed Liveness Probe, or a preStop hook timeout.
    - *Disambiguation:* The engine will cross-reference the containerStatuses[].lastState.terminated.reason. If the reason is OOMKilled, it maps to Infrastructure (Memory). If the reason is Error but the logs contain 'Liveness probe failed', it maps to Application (Health).
- Exit Code 1 (Generic Error):
    - *Ambiguity:* The 'catch-all' for any application-level crash.
    - *Disambiguation:* The engine will trigger a Log-Pattern Scan on the last 50 lines of the stderr stream. It will look for high-confidence signatures like ModuleNotFoundError (Python), Segmentation fault (C++), or java.lang.NullPointerException to re-categorize the 'Generic' error into a specific subtype.

- Exit Code 127 (Command Not Found):
    - *Ambiguity:* Can be a missing binary in the image or a typo in the pipeline_spec.
    - *Disambiguation:* By inspecting the KFP Runtime Spec against the image's ENTRYPOINT, the engine will distinguish between an **Image Configuration Error** (missing tool) vs. a **Workflow Definition Error** (incorrectly passed command-line argument)."

Example:

If:

k8sReason == "OOMKilled" → classify as ResourceLimitError

→ Suggest increasing memory limits

This rule-based schema avoids random log summarization and ensures predictable behavior.

3. API Extension

Extend Run Details API response:

Add optional field:

```
errorSummary: {
  category,
  rootCause,
  confidence,
  suggestedFix
}
```

If classification fails → fallback to raw logs only.

This ensures backward compatibility

4. UI Integration (Frontend)

Modify:

frontend/src/pages/RunDetails.tsx

- Add new component: ErrorSummaryPanel.tsx

Panel behavior:

- Visible only when run.status == Failed
- Displays structured summary at top
- Collapsible detailed view
- Link to full logs

Design principles:

- No clutter
- Clear visual separation
- Lightweight rendering
- Progressive disclosure

Logs fetched lazily to prevent performance impact.

5. Rule prioritization strategy

If multiple rules match the same log segment, the classifier will:

1. Rank by severity weight (High > Medium > Low)
2. Break ties using confidence score
3. Prefer structured-signal matches (exit code / pod reason) over regex-only matches
4. Surface secondary matches in expandable "Additional Possible Causes" section

This prevents ambiguous classification while maintaining transparency.

6. Root-Cause Prioritization Logic

If multiple nodes fail:

- Prioritize earliest terminal failure.
- Ignore cascading downstream failures.
- Rank by severity (Infrastructure > Dependency > Runtime).
- This prevents misleading summaries.

7. Safety & Misclassification Guard

To prevent misleading summaries:

- The ErrorSummaryBanner will always be expandable, allowing users to view the full raw log.
- If classification confidence < 0.60, no banner will be rendered.
- If multiple high-confidence matches occur, the primary cause will be shown while secondary matches appear under "Additional Possible Causes."
- A disclaimer line will be shown:

"Classification is heuristic-based and may not reflect the exact root cause. Refer to raw logs for complete details."

This ensures transparency and prevents overconfidence in automated classification

8. Extensibility

- Classifiers defined in structured configuration.
- Easy to add new error patterns.
- Future support for:

    1. Plugin architecture
    2. AI-assisted summarization
    3. Telemetry-based frequent failure detection

Note: AI-assisted debugging is not part of the scope of this GSoC project. The current implementation focuses exclusively on deterministic rule-based classification. AI-assisted approaches are mentioned only as a potential long-term extension.
While the current scope focuses on deterministic rule-based classification, the architecture is designed for future extensions such as plugin-based rule packs, telemetry-driven prioritization, and AI-assisted summarization.

Success Metrics

Unlike single-signal approaches that rely only on exit codes or raw logs, this system correlates pod status, termination reasons, and log patterns to reduce false positives. This improves classification precision and prevents misleading summaries in ambiguous cases such as exit code 137.

- Log parsing latency will be maintained under 100ms.
- 100% detection of standard K8s exit codes (137, 143, 1).
- Support for 15+ common ML exception signatures
- ≥90% unit test coverage.
- Reduce clicks-to-insight from 6 steps to 1.

Performance Benchmarking Methodology

Classification performance will be benchmarked using:

- Log sizes: 50KB, 250KB, 500KB, 1MB
- Rule counts: 10, 25, 50
- Environment: Chrome DevTools Performance profiling
- Measurement: average classification time across 100 iterations

Worst-case time complexity:
O(n) where n = rule count (bounded and small).
Target:
<100ms classification for 1MB log with 25 rules on a mid-range developer machine.

Results will be documented in the repository

To ensure UI responsiveness, log analysis is limited to the final 200–500 lines of the log output rather than processing the entire log stream.
This segment typically contains the terminal failure signals required for classification.

The analysis is executed asynchronously in the frontend so that classification runs in the background and does not block UI rendering in the RunDetails page.

Implementation Timeline

Each sprint deliverable will be submitted as a small, reviewable PR aligned with Kubeflow's incremental integration model. This reduces review overhead and enables faster, safer upstream integration.

Weeks 1–2: Core Classifier Foundation

- Implement ErrorRule interface.
- Create initial errorRules.ts registry.
- Implement classifyError() engine.
- Add unit tests for rule evaluation.
- Validate deterministic rule ordering.

Deliverable:

Working rule engine with at least 5 core Kubernetes error rules + unit tests.

Weeks 3–4: Signal Extraction & Log Handling

- Integrate classifier with LogViewer tail extraction (≤1MB).
- Parse pod status + exit codes.
- Implement structured signal normalization.
- Add confidence scoring mechanism.
- Benchmark classification latency.

Deliverable (Midterm Evaluation Target):

CLI-level classification working inside RunDetails with console output + performance benchmark results (<100ms target).

Weeks 5–6: UI Integration Phase

- Implement ErrorSummaryPanel.tsx.

- Inject into RunDetails.tsx.
- Add conditional rendering (Failed status only).
- Implement expandable raw log fallback.
- Ensure no regression in existing UI behavior.

Deliverable:

Visible Error Summary Banner in failed pipeline runs.

Weeks 7–8: Prioritization & Safety Layer

- Implement rule prioritization strategy.
- Add secondary match rendering.
- Add misclassification guard (confidence threshold).
- Add transparency disclaimer.
- Improve UI accessibility (ARIA labels, keyboard nav).

Deliverable:

Stable UI with prioritized and safe classification rendering.

Weeks 9–10: Testing & Hardening

- Add Jest unit tests for all rules.
- Achieve ≥90% test coverage for classifier.
- Add Cypress E2E tests for failure scenarios.
- Test against simulated OOM, ImagePull, CrashLoop cases.
- Profile performance under 1MB logs.

Deliverable:

Test suite + documented benchmark results.

Week 11: Documentation & Extensibility

- Document rule extension process.
- Create CONTRIBUTING_ERRORS.md.
- Add inline documentation.
- Clean up code structure.
- Prepare demo recording.

Deliverable:

Complete developer documentation + extensibility guide.

Week 12: Buffer & Final Polish

- Address mentor feedback.
- Refactor based on review comments.
- Fix edge cases.
- Final performance validation.
- Submit final PR(s).

Deliverable:

Production-ready prototype with pull requests prepared for upstream review and incremental integration.

Definition of Done (DoD)

Each 2-week sprint will deliver a stable and reviewable increment aligned with Kubeflow contribution standards.

A sprint is considered complete only when:

- All planned features for the sprint are fully implemented and functionally validated
- Unit tests are added for new logic and all tests pass successfully
- Code follows Kubeflow frontend conventions (TypeScript, React patterns, modular structure)
- No regression is introduced in existing Kubeflow Pipelines UI behavior
- Feature is verified within the RunDetails workflow using real or simulated failure scenarios
- UI changes are responsive, accessible, and do not introduce rendering delays
- Performance remains within acceptable limits (no blocking during log processing)
- Code is clean, documented, and ready for mentor review or pull request submission

This ensures that every milestone produces production-quality, incrementally integrable contributions.

Risks & Mitigation

- Performance safeguarded via tail-only parsing & memoization.
- Ambiguity handled using priority-based confidence scoring.
- UI clutter avoided through progressive disclosure design.
- No Argo execution changes—maintains architectural integrity

Fallback Plan:

If classification confidence is low or signals are ambiguous:

-> No summary will be rendered

- > System falls back to raw logs

- > Relevant log segments are highlighted for user guidance

Expected Technical Outcome

This feature introduces a clear error summary layer in Kubeflow Pipelines, transforming raw Kubernetes execution signals into actionable debugging insights.

It bridges the gap between infrastructure-level failure signals and ML developer usability.

Expected Impact

This project enhances observability in Kubeflow Pipelines by transforming low-level Kubernetes signals into concise summaries, human-readable insights. The extensible rule-based architecture also establishes a foundation for future enhancements, including additional failure patterns and advanced debugging assistance.

1. Reduced Debugging Time

By automatically identifying the failing step, classifying the error type, and extracting the most relevant log signals, users will no longer need to manually inspect multiple workflow nodes and dense logs. This will substantially reduce Time-to-Insight during pipeline failures.

By reducing debugging from multiple manual steps to a single actionable insight, the system lowers cognitive load and accelerates iteration cycles, making Kubeflow more accessible to ML practitioners.

2. Improved Developer Productivity

Instead of scanning raw logs, developers will receive:

- Categorized error summaries
- Clear root-cause indication
- Suggested remediation hints

This reduces cognitive overhead and accelerates iteration cycles in ML experimentation.

3. Lower Onboarding Barrier

New contributors and ML practitioners often lack deep Kubernetes debugging expertise. Structured error abstraction will make failure diagnosis more accessible and intuitive, improving onboarding experience.

4. Increased Reliability Perception

Providing actionable failure summaries enhances usability and reduces operational friction, strengthening Kubeflow Pipelines as a production-grade ML orchestration platform.

5. Scalable & Extensible Debugging Framework

The rule-based classification engine creates a foundation for:

- Adding new error classifiers
- Supporting additional failure patterns
- Future AI-assisted debugging capabilities

This ensures long-term ecosystem value beyond the initial implementation.

6. Measurable Success Indicators

The impact of this feature can be evaluated through:

- Reduced average debugging duration
- Faster identification of root causes
- Developer feedback from maintainers and contributors
- Increased clarity in pipeline failure reports

Prior Contributions

I began contributing to Kubeflow Pipelines prior to submitting this proposal to build architectural familiarity and demonstrate sustained engagement. Actively engaged with Kubeflow maintainers through PR reviews and incorporated feedback such as UI improvements and validation changes.

1. Documentation Improvement – Backend Architecture Clarification

PR #12871

Contributed a *Backend Component Overview* section to the backend README to improve contributor onboarding and architectural clarity.

This addition:

- Clearly explains the API Server, Driver, Launcher, Persistence Agent, and Metadata components
- Reduces ramp-up time for new contributors
- Improves system-level comprehension of execution flow

This work required tracing the Go backend and understanding control-plane responsibilities in relation to Argo Workflows.

2. Frontend UX Enhancement – RunDetails Loading Experience

PR #12873

Improved the RunDetails page by replacing the plain loading text with a centered Material-UI CircularProgress spinner.

Key aspects of this contribution:

- Enhanced perceived responsiveness during run metadata fetch
- Preserved backend contracts and execution semantics
- Modified only RunDetails.tsx, maintaining strict architectural containment
- Ensured no changes to APIs, persistence logic, or workflow execution

This contribution demonstrates:

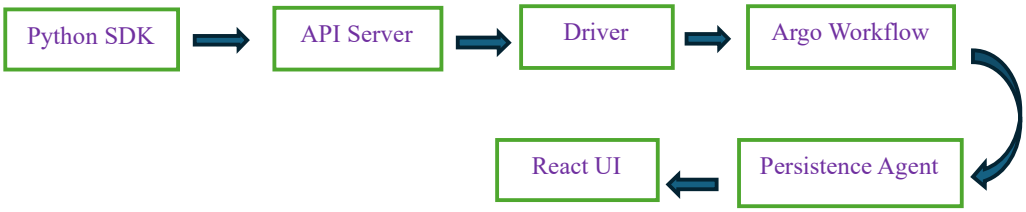
- Direct familiarity with the React + TypeScript frontend
- Comfort navigating Kubeflow's UI rendering logic
- Ability to introduce user-facing improvements without introducing regression risk

---

3. Development Environment & Architectural Study

Beyond PRs, I have:

- Set up a full local Kubeflow Pipelines development environment using **Kind + Argo Workflows**
- Reproduced real pipeline failure scenarios to understand debugging pain points
- Traced the full execution lifecycle:



- Reviewed key frontend modules relevant to this proposal:
  - RunDetails.tsx
  - LogViewer.tsx
  - NodeDetailsPanel.tsx

This groundwork ensures the proposed Error Classification Engine integrates strictly at the frontend layer without altering workflow execution semantics

4. Prototype Implementation – Error Classification UI Demo Developed a frontend prototype that classifies Kubernetes pod failures and provides root cause with suggested fixes.

GitHub: https://github.com/RATNAPRADEEP/Kubeflow-ui-error-analysis

Time Commitment & Availability

During the GSoC coding period I will dedicate approximately 30–35 hours per week to this project.

I do not have any conflicting academic or professional commitments during this period and will maintain regular communication with mentors through Kubeflow Slack,GitHub discussions, and weekly progress updates.

This availability ensures consistent progress and timely completion of the project milestones.

Communication Plan

- Weekly progress updates on Kubeflow Slack
- Regular pull requests for incremental feedback
- Active discussions with mentors during design and implementation
- Quick response to review comments and suggested changes

This ensures transparency and continuous alignment with project expectations.

Pre- GSoC Contribution Plan

Before the official coding period, I will continue contributing small, incremental pull requests to Kubeflow Pipelines, focusing on frontend improvements and minor fixes in the RunDetails UI. This will help deepen my familiarity with the codebase and align my implementation approach with maintainer expectations.

Why Me?

Built a working prototype demonstrating error classification (GitHub link above)

I approach this project with a systems-level mindset shaped by both engineering and testing experience. I have already validated this idea through a working prototype and initial contributions, reducing implementation risk and enabling faster delivery during GSoC.

By combining my SDET background with a commitment to **Developer Experience (DevEx) Modernization**, I aim to deliver an intelligent debugging layer that reduces cognitive load for data scientists while maintaining the strict architectural integrity required of KFPv2.

As a former Software Development Engineer in Test (SDET), I have worked extensively with distributed applications, CI/CD systems, and multi-layer debugging workflows. This background trained me to reason about failure modes across boundaries—API layer, container runtime, and orchestration layer—while minimizing regression risk.

My contribution philosophy emphasizes:

- Architectural containment over invasive change
- Measurable performance guarantees
- High test coverage and regression safety
- Transparent UX that avoids overconfident automation

Before and during GSoC, I will contribute incremental pull requests, actively engage with maintainers, and validate all changes with unit and E2E tests aligned with Kubeflow's standards.

As part of community interaction, I have engaged with Kubeflow maintainers through GitHub PR discussions and reviews, incorporated feedback on frontend integration and UI improvements, and actively followed discussions related to the RunDetails UI and debugging workflows.

With an existing prototype, prior contributions, and a clear incremental plan, I am well-positioned to deliver a reliable and impactful improvement to Kubeflow Pipelines during GSoC.